Architecture

Scaling an agent from 50 tools to 1,000+ without losing tool-selection accuracy

---

1. The problem

An LLM given a handful of tools picks between them reliably. The same model given hundreds degrades, in three distinct ways. **Tool confusion:** it selects a semantic neighbour, for example listing invoices when asked to send one. **Argument hallucination:** it borrows a field name from an adjacent schema, or invents an identifier it was never given. **Context saturation:** every schema in the prompt costs tokens and latency, whether or not it is relevant.

The failure is not the model's reasoning but the size of its choice set. So the design goal is not a better prompt; it is to make the number of registered tools irrelevant to what the model sees on any given turn.

2. The approach: bounded candidate retrieval

Tool *storage* is decoupled from tool *reasoning*. The catalog may hold any number of tools. A retrieval stage narrows it to a small shortlist, and only that shortlist is ever offered to the model.

> **KEY POINT**
> Catalog size affects the retrieval index, not the prompt. Whether 60 tools are registered or 6,000, the planner is handed roughly ten candidate schemas. The prompt stays the same size, so accuracy and cost stay flat as the catalog grows.

Two-stage retrieval

Implemented in `app/services/tool_retriever.py`:

- **Stage 1 - domain scoping.** The query is classified into an operational domain (invoicing, payments, disputes, reporting, and so on) from keyword patterns. This narrows the search space before any vector work.
- **Stage 2 - hybrid search.** A dense vector search over Qdrant runs alongside a sparse keyword score. Dense search generalises across phrasing; sparse matching catches exact identifiers and API names that embeddings tend to blur.
- **Fusion.** The two rankings are merged with Reciprocal Rank Fusion, so a result ranked well by either method survives without needing a tuned score weighting.
- **Dependency expansion.** Prerequisite tools are pulled in alongside their dependants, so selecting `send_invoice` also surfaces `create_draft_invoice`.
- **Relaxation.** If domain-scoped search returns too little, the domain filter is dropped and the search repeats, so a misclassified query degrades rather than fails.

Dual-index storage

Each tool exists in two places, kept in sync by `app/services/tool_registry.py`. The relational row is the authoritative contract: endpoint, parameter schemas by location, risk tier, approval policy, permitted roles, and prerequisite edges. The vector point carries an embedded capability summary used only for discovery. Retrieval finds candidates by vector; execution and governance always read the relational contract.

3. Reasoning over the shortlist

The shortlist is translated into Gemini function declarations, one per candidate, carrying each parameter's type, description and required flag (`app/services/llm_service.py`). The model chooses one tool and fills its arguments. Three constraints are enforced in code rather than asked for in the prompt:

- **The shortlist is a hard boundary.** A returned tool name outside the candidate set is discarded before it can reach the execution engine.
- **One tool per turn.** The planner selects a single next action; sequences emerge from the loop, not from a plan the model commits to up front.
- **Missing arguments are asked for, not invented.** When a required parameter cannot be grounded in the request or in a previous step's output, the model replies with text requesting it instead of calling the tool.

Answers are then synthesised from the execution trace, so every figure in a reply traces to a tool result rather than to the model's prior.

4. Framework choices and trade-offs

Choice	Why	Trade-off accepted
LangGraph	An explicit state machine with typed transitions. Each stage is a named node, so the path a turn took is inspectable, and the human-approval pause is a first-class graph feature (interrupt plus checkpointer) rather than application plumbing.	More wiring than a prebuilt agent loop. Nodes and edges are declared by hand.
Not CrewAI / AutoGen	Role-playing multi-agent frameworks solve collaboration between agents. The problem here is tool selection within one agent, which their abstractions do not address.	No built-in multi-agent delegation if the scope later needs it.
Not a bare ReAct loop	Free-form loops are hard to bound, audit, or pause mid-flight. A financial action needs a deterministic point at which approval is enforced.	Less emergent flexibility; new behaviour means a new node or edge.
Qdrant	Payload filtering lets domain scoping run inside the vector query, and it runs in-memory for local evaluation with no external service.	In-memory mode loses the index on restart; the catalog is re-seeded.
Hybrid retrieval over pure vector search	Embeddings blur near-identical API names. Sparse matching recovers exact tokens such as an endpoint or an identifier.	Two searches per turn instead of one.
FastAPI + Streamlit over one process	A REST boundary means authorisation and role checks are enforced server-side and the UI cannot bypass them.	Two processes to run.

5. Scaling from 50 to 500+ tools

Adding services changes the catalog, not the prompt. A new service is ingested into the registry, indexed, and assigned a namespace; retrieval then scopes to the namespaces an intent implies. The planner's input stays bounded at roughly ten candidates.

Dimension	Behaviour as the catalog grows
Prompt size	Flat. Bounded by top-k, not by catalog size.
Retrieval cost	Sub-linear. Vector search over an indexed collection, scoped by domain payload filter.
Governance	Per-tool. Risk tier and approval policy are columns on the tool, so a new mutating endpoint inherits the approval gate automatically.
Namespacing	Hierarchical (for example paypal.invoicing, communication.email), so cross-service collisions do not occur.
Onboarding a service	Ingest a collection, register, index. No prompt or code change.

6. Degraded operation

Every external dependency has a fallback, so the platform remains demonstrable without credentials or network access.

Dependency	Fallback
Gemini (reasoning)	A deterministic rule-based planner and template responses. The pipeline, retrieval, governance and audit trail behave identically; only argument inference is weaker.
Gemini (embeddings)	A local deterministic hash embedding, so indexing and retrieval still function offline.
Qdrant server	In-memory client.
PostgreSQL	SQLite via aiosqlite.
PayPal sandbox	Mock dispatch returning realistic response shapes.

Dependency	Fallback
SQLite checkpointer	In-memory checkpointer; approvals then do not survive a restart, and this is logged.

LIMITATION

The fallback planner recognises only the intents encoded in its rules. It exists so the system can be evaluated without an API key, not as an equivalent substitute for the model. Configure GEMINI_API_KEY to exercise the designed path.